APPENDIX A: CONCEPT-MASTERY ESTIMATOR PROMPT TEMPLATE

This appendix provides the complete prompt structure used for the Concept-Mastery Estimator.

A.1: System Prompt

You are an experienced educational assessment expert. Your task is to evaluate a student's mastery level of a specific knowledge component based on their exam performance data.

Focus on analysing:

- Overall performance patterns across all questions
- Performance consistency and stability
- Handling of questions with different difficulties
- Behavioural signals (confidence, hint usage, hesitation)
- Performance on questions involving multiple knowledge components

A.2: Concept-Mastery Estimator User Prompt Structure

— ASSESSMENT CONTEXT —
Student ID: 144
Knowledge Component: 'Lossless Join'
Description: To disallow the possibility of generating spurious tuples when a NATURAL JOIN operation is applied to the relations after decomposition.

— EXAM PERFORMANCE RECORDS FOR 'Lossless Join' —
Total questions answered: 4

[Question 1]
Question ID: 130
Question Content: $R = \{A, B, C\}$ and $\Sigma = \{A \rightarrow B,\ B \rightarrow C\}$. The decomposition of R into $R_1 = \{A, B\}$ and $R_2 = \{B, C\}$.
Answer Choices:

- Does not fulfil the lossless join property but fulfils the dependency preserving property.
- Fulfils both lossless join and dependency preserving properties. [Correct Answer] ← [Student's Choice]
- Fulfils neither the lossless join property nor the dependency preserving property.
- Fulfils the lossless join property but does not fulfil the dependency preserving property.

Result: ✓ Correct

[Question 2]
Question ID: 131
Question Content: $R = \{A, B, C\}$ and $\Sigma = \{A \rightarrow B,\ B \rightarrow C,\ A \rightarrow C\}$. The decomposition of R into $R_1 = \{A, B\}$ and $R_2 = \{A, C\}$.
Answer Choices:

- Does not fulfil the lossless join property but fulfils the dependency preserving property. ← [Student's Choice]
- Fulfils both lossless join and dependency preserving properties.
- Fulfils neither the lossless join property nor the dependency preserving property.
- Fulfils the lossless join property but does not fulfil the dependency preserving property. [Correct Answer]

Result: ✗ Incorrect

[Question 3]
Question ID: 132
Question Content: $R = \{A, B, C\}$ and $\Sigma = \{A \rightarrow B,\ B \rightarrow C,\ A \rightarrow C\}$. The decomposition of R into $R_1 = \{A, B\}$ and $R_2 = \{B, C\}$.
Answer Choices:

- Does not fulfil the lossless join property but fulfils the dependency preserving property.
- Fulfils both lossless join and dependency preserving properties. [Correct Answer]
- Fulfils neither the lossless join property nor the dependency preserving property.
- Fulfils the lossless join property but does not fulfil the dependency preserving property. ← [Student's Choice]

Result: ✗ Incorrect

[Question 4]
Question ID: 143
Question Content: Which of the following is correct?
Answer Choices:

- The BCNF decomposition is always unique for any R and Σ.
- The 3NF decomposition is always unique for any R and Σ.
- There exists an algorithm that can always generate a lossless and dependency-preserving decomposition into 3NF. [Correct Answer] ← [Student's Choice]
- There exists an algorithm that can always generate a lossless and dependency-preserving decomposition into BCNF.

Result: ✓ Correct

— ASSESSMENT TASK —
Based on the exam performance records above, evaluate the student's mastery level of this knowledge component.
Choose **ONE** mastery level from: [Novice, Developing, Proficient, Mastered]
Level Definitions:

- Novice: Limited understanding, frequent errors, low confidence
- Developing: Partial understanding, inconsistent performance, needs improvement
- Proficient: Solid understanding, mostly correct answers, occasional mistakes on complex questions
- Mastered: Comprehensive understanding, consistently correct, high confidence across all difficulty levels

Provide:
(1) Your chosen mastery level
(2) Detailed rationale citing specific question performances and Behavioural patterns
(3) Actionable suggestions for improvement (if applicable)

— OUTPUT FORMAT —
Please structure your response exactly as follows:
Mastery Level: <Your chosen level>
Rationale: <Detailed explanation with specific evidence from the exam records>
Suggestions: <Actionable recommendations for the student>

A.3: Concept-Mastery Estimator Response

LLM Response:
Mastery Level: Developing

Rationale: The student demonstrates partial understanding of the 'Lossless Join' knowledge component but exhibits inconsistency in applying the concept, particularly when distinguishing between lossless join and dependency preservation in similar-looking decompositions. The student correctly answered two out of four questions (Q1 and Q4), showing they can apply the concept in simpler or more abstract contexts. In Q1, they correctly identified that the decomposition $R_1 = \{A, B\}$, $R_2 = \{B, C\}$ under $\Sigma = \{A \to B,\ B \to C\}$ is both lossless and dependency-preserving — this indicates a correct grasp of the intersection condition ($R_1 \cap R_2 = \{B\}$, which functionally determines R_2 via $B \to C$) and the ability to assess dependency preservation. However, in Q2, the student incorrectly chose that the decomposition does *not* have the lossless join property, when in fact $R_1 \cap R_2 = \{A\}$, and $A \to B$ and $A \to C$ imply A is a superkey, making the decomposition lossless — this reveals a gap in recognizing superkey-based losslessness. Similarly, in Q3, the student believed the decomposition was only lossless, but not dependency-preserving, while the correct answer includes both properties; this suggests confusion about whether all functional dependencies are preserved across projections, especially $B \to C$ being preserved in R_2. The one strength is in Q4, where the student correctly selected that 3NF allows for a guaranteed lossless and dependency-preserving decomposition — showing awareness of theoretical guarantees at a higher level. However, the repeated errors on structurally similar problems (Q2 and Q3) indicate unstable or incomplete internalization of the criteria for lossless joins and dependency preservation.

Suggestions: Focus on practicing the formal conditions for lossless join decomposition: ensure that the intersection of decomposed schemas is a superkey for at least one relation. Work through multiple examples where the intersection is a key versus a non-key attribute, and explicitly verify each functional dependency's preservation in projected relations. Use step-by-step worksheets to check both properties separately. Review how closure of attributes and canonical covers influence dependency preservation. Revisit prerequisite topics like normalisation and functional dependency inference to strengthen foundational reasoning. Consider creating comparison tables of similar decompositions (e.g., sharing B vs. sharing A as the intersection) to highlight subtle differences in outcomes.

APPENDIX B: TUTORING-CONTENT GENERATOR PROMPT TEMPLATE

This appendix provides the complete prompt structure used for personalised tutoring content generation.

B.1 System Prompt

You are a personalised Tutoring-Content Generator. Your goal is to help students consolidate their weak knowledge components. For each knowledge component, you should:

(1) provide 3–5 concise learning points,
(2) summarise common misconceptions, and
(3) give detailed explanations in **Chinese** based on the three provided practice questions (including their correct answers).

B.2 User Prompt (Case Study: Lossless Join, Student ID: 144)

Student ID: 144

— **Knowledge components requiring focused tutoring** —

[**Knowledge Component**] Lossless Join
Description: To disallow the possibility of generating spurious tuples when a NATURAL JOIN operation is applied to the relations after decomposition.

Example 1 (Question ID: 131)
Question: $R = \{A, B, C\}$ and $\Sigma = \{A \to B,\ B \to C,\ A \to C\}$. The decomposition of R into $R_1 = \{A, B\}$ and $R_2 = \{A, C\}$.
Options:
A. Does not fulfil the lossless join property but fulfils the dependency preserving property.
B. Fulfils both lossless join and dependency preserving properties.
C. Fulfils neither the lossless join property nor the dependency preserving property.
D. Fulfils the lossless join property but does not fulfil the dependency preserving property.
Correct answer: D – Fulfils the lossless join property but does not fulfil the dependency preserving property.

Example 2 (Question ID: 143)
Question: Which of the following is correct?
Options:
A. The BCNF decomposition is always unique for any R and Σ.
B. The 3NF decomposition is always unique for any R and Σ.
C. There exists an algorithm that can always generate a lossless and dependency-preserving decomposition into 3NF.
D. There exists an algorithm that can always generate a lossless and dependency-preserving decomposition into BCNF.
Correct answer: C – There exists an algorithm that can always generate a lossless and dependency-preserving decomposition into 3NF.

Example 3 (Question ID: 130)
Question: $R = \{A, B, C\}$ and $\Sigma = \{A \to B,\ B \to C\}$. The decomposition of R into $R_1 = \{A, B\}$ and $R_2 = \{B, C\}$.
Options:
A. Does not fulfil the lossless join property but fulfils the dependency preserving property.
B. Fulfils both lossless join and dependency preserving properties.
C. Fulfils neither the lossless join property nor the dependency preserving property.
D. Fulfils the lossless join property but does not fulfil the dependency preserving property.
Correct answer: B – Fulfils both lossless join and dependency preserving properties.

Task: For the above knowledge component, first provide 3–5 key learning points and common misconceptions. Then, for each example question, give a detailed explanation in **Chinese**, including: problem-solving idea, key steps, why the correct answer is correct, and typical pitfalls or sources of error.

Output format example:
Knowledge component: <name>
Key points: 1) … 2) … 3) …
Misconceptions: 1) … 2) …
Explanation – Example 1: …

Explanation – Example 2: ...
Explanation – Example 3: ...

B.3 LLM Response (Case Study: Lossless Join)

Knowledge component: Lossless Join

Key points:

(1) The core idea of a lossless join decomposition is that, after decomposing a relation into smaller relations, performing a NATURAL JOIN on the decomposed relations should exactly reconstruct the original relation without producing spurious (extra or incorrect) tuples.

(2) For a binary decomposition of R into R_1 and R_2, a sufficient and commonly used condition for losslessness is that the intersection attributes $R_1 \cap R_2$ functionally determine all nonshared attributes of at least one side, i.e.,

$$R_1 \cap R_2 \to (R_1 - R_2) \quad \text{or} \quad R_1 \cap R_2 \to (R_2 - R_1).$$

(3) Intuitively, the intersection $R_1 \cap R_2$ should act as a *superkey* for at least one of the decomposed relations, so that the join does not freely recombine tuples.

(4) Lossless join and dependency preservation are two different dimensions: losslessness is about reconstructing data without distortion, while dependency preservation is about being able to enforce the original functional dependencies locally on the decomposed relations.

(5) A standard synthesis algorithm for 3NF can guarantee both lossless join and dependency preservation, whereas BCNF decompositions may have to sacrifice dependency preservation in some cases in order to remove all redundancy.

Common misconceptions:

(1) Believing that as long as we can join the decomposed relations back together, the decomposition is necessarily lossless, without checking whether spurious tuples might be introduced.

(2) Confusing lossless join with dependency preservation, and assuming that if one holds, the other must hold automatically.

(3) Ignoring the role of the intersection attributes and judging losslessness only by intuition, rather than checking whether the intersection is a superkey for one of the fragments.

(4) Assuming that BCNF is always strictly better than 3NF and can always guarantee both losslessness and dependency preservation.

Explanation – Example 1 (Question ID: 131):

In this question, we have $R = \{A, B, C\}$, $\Sigma = \{A \to B,\ B \to C,\ A \to C\}$, and the decomposition into $R_1 = \{A, B\}$ and $R_2 = \{A, C\}$.

Step 1: Check lossless join.

The intersection is $R_1 \cap R_2 = \{A\}$. We examine whether A functionally determines the nonshared attributes of at least one side:

$$R_1 - R_2 = \{B\}, \quad \text{and we know } A \to B \text{ from } \Sigma.$$

Thus $A \to (R_1 - R_2)$ holds, which satisfies the standard lossless join condition. Therefore, this decomposition is *lossless*.

Step 2: Check dependency preservation.

The original set of dependencies is $\Sigma = \{A \to B,\ B \to C,\ A \to C\}$.

- $A \to B$ involves only A and B, both of which lie in R_1, so this dependency can be enforced locally in R_1.
- $A \to C$ involves A and C, both in R_2, so it can be enforced locally in R_2.
- $B \to C$ is more problematic: B appears in R_1 (and as an intersection attribute), C appears only in R_2, and there is no single fragment that contains both B and C in a way that allows this dependency to be checked locally.

Projecting Σ onto R_1 yields essentially $A \to B$, and onto R_2 yields $A \to C$. From these projected dependencies we cannot derive $B \to C$. Therefore, enforcing $B \to C$ would require joining R_1 and R_2, which means that this dependency is *not* preserved.

Overall, the decomposition is lossless but not dependencypreserving, so option D ("Fulfils the lossless join property but does not fulfil the dependency preserving property") is correct. A typical pitfall for students is to notice that A is a strong determinant and then incorrectly assume that *all* dependencies are preserved, without explicitly checking where $B \to C$ goes after decomposition.

Explanation – Example 2 (Question ID: 143):

This question compares theoretical properties of BCNF and 3NF decompositions. The correct answer is C. We evaluate each option:

- **A.** "The BCNF decomposition is always unique for any R and Σ." This is false. BCNF decompositions are not unique: different orders of choosing violating dependencies and different decomposition paths can lead to different final schemas.

- **B.** "The 3NF decomposition is always unique for any R and Σ." This is also false. 3NF decompositions depend on the chosen minimal cover and grouping of dependencies; multiple valid decompositions can exist.
- **C.** "There exists an algorithm that can always generate a lossless and dependencypreserving decomposition into 3NF." This is true. The standard 3NF synthesis algorithm, based on a minimal cover of Σ, constructs relation schemas in such a way that the resulting decomposition is guaranteed to be both lossless and dependency-preserving, and all resulting relations are in 3NF.
- **D.** "There exists an algorithm that can always generate a lossless and dependencypreserving decomposition into BCNF." This is false. For some sets of functional dependencies, it is impossible to have a BCNF decomposition that preserves all dependencies; to reach BCNF one may need to sacrifice dependency preservation.

This example emphasises a key conceptual contrast: although BCNF removes more redundancy, it does not always allow dependency preservation, whereas 3NF admits an algorithmic decomposition that guarantees both lossless join and preservation of all functional dependencies. This is closely related to how we reason about lossless join versus dependency preservation in practice.

Explanation – Example 3 (Question ID: 130):
Here we have $R = \{A, B, C\}$, $\Sigma = \{A \to B,\ B \to C\}$, and the decomposition into $R_1 = \{A, B\}$ and $R_2 = \{B, C\}$.
Step 1: Check lossless join.
The intersection is $R_1 \cap R_2 = \{B\}$. We examine:

$$R_2 - R_1 = \{C\}, \quad \text{and from } \Sigma \text{ we have } B \to C.$$

Thus $B \to (R_2 - R_1)$ holds, satisfying the lossless join condition. Therefore, the decomposition is *lossless*.
Step 2: Check dependency preservation.
We check whether each dependency in Σ can be enforced within some fragment:
- $A \to B$ involves attributes A and B, both of which lie in R_1, so this dependency is preserved in R_1.
- $B \to C$ involves B and C, both of which lie in R_2, so this dependency is preserved in R_2.

Since each original dependency is fully contained in at least one of the fragments, the decomposition preserves all dependencies in Σ. Hence this decomposition is both lossless and dependencypreserving, and the correct answer is B.
Conceptually, this is a classic "chain" example where $A \to B \to C$ and the decomposition cuts at the middle attribute B. Attribute B acts as the link: it makes the join lossless (through $B \to C$) and at the same time allows each dependency to live entirely inside one fragment. Students often confuse this case with the previous example, so deliberately comparing the two (intersection A vs. intersection B and whether the intersection is a superkey for one side) is a very effective way to deepen understanding of lossless join conditions.

APPENDIX C: STUDENT AGENT PROMPT TEMPLATE

This appendix provides the complete prompt structure used for student behaviour simulation, including the integration of two prompt-based state representations: concept-mastery summaries (longer-term proficiency) and tutoring summaries (recently reviewed practice content).

Prompt composition of the four experimental modes

Mode	C.1 Learner Profile	C.2-0 USER PROMPT	C.2-1 Mastery Summary	C.2-2 Tutoring Summary
Baseline (P_u-Only)	✓	✓	✗	✗
Mastery Only($\mathcal{M}_u$+P_u)	✓	✓	✓	✗
Tutoring Only($\mathcal{T}_u$+P_u)	✓	✓	✗	✓
Combined ($\mathcal{M}_u$+$\mathcal{T}_u$+P_u)	✓	✓	✓	✓

C.1 Learner Profile(System Prompt)

You ARE a student with these learning characteristics:

📚 Your Learning Profile:
- Activity Level: medium - You practice occasionally when needed
- Knowledge Breadth: low - You stick to familiar topics you feel comfortable with
- Typical Success Rate: high
- Problem-Solving Ability: good
- Most Comfortable Topic: Functional dependencies

🎯 How to Respond:
1. Think and answer as THIS student would - based on YOUR actual abilities and experiences
2. Be honest about your confidence level - don't overestimate or underestimate yourself

3. When predicting performance, reflect on YOUR past experiences with similar problems
4. If you're unsure or haven't mastered a concept, it's okay to predict 'No' - be realistic
5. Your responses should reflect your genuine thought process as this student

C.2-0 USER PROMPT (used with C.2-1 Mastery Summary / C.2-2 Tutoring Summary)

=== 📝 The Question in Front of You ===
Question: $R = \{A, B, C\}$ and $\Sigma = \{A \to B,\ B \to C,\ A \to C\}$. The decomposition of R into $R_1 = \{A, B\}$ and $R_2 = \{B, C\}$.

Answer Choices:
A. Does not fulfil the lossless join property but fulfils the dependency preserving property.
B. Fulfils both lossless join and dependency preserving properties.
C. Fulfils neither the lossless join property nor the dependency preserving property.
D. Fulfils the lossless join property but does not fulfil the dependency preserving property.

Insret here with C.2-1 Mastery Summary / C.2-2 Tutoring Summary
=== 🤔 Now, Think Through This Question as This Student ===

Task 1: Honestly predict - will you get this right?
(Based on your knowledge and confidence about 'Lossless Join')
Think to yourself:
- Did I just review this topic? If so, I should feel more confident!
- Do the example problems I studied help me understand this question?
- Am I confident I can apply what I just learned?

Your honest prediction (Yes/No):

Task 2: What topic does this question test?
(Based on what you see, which concept is this about?)
Options: Concurrent Transactions, Lossless Join, Serializable
Your identification:

Reasoning: How would you approach and solve this?
(Think about what you just reviewed - can you apply any of those concepts or methods here?)
(If this is similar to the example problems, follow that solving approach)
Your work:

Task 3: What is your final answer choice?
(Select the option you believe is correct)
Available options: A, B, C, D
Your choice:

Output format:
Task1: <Answer>
Task2: <Answer>
Reasoning: <Answer>
Task3: <Answer>

C.2-1 Mastery Summary

Topic: Lossless Join

=== 🧠 Your Concept-Mastery Summary for This Topic ===
Based on your accumulated learning experience:
📌 You're looking at: Lossless Join
You feel you are at: Developing
Your confidence level: ⚡ Moderate Confidence - You have basic understanding but may struggle

You've noticed: The student demonstrates partial understanding of the 'Lossless Join' knowledge component but exhibits inconsistency in applying the concept, particularly when distinguishing between lossless join and dependency preservation in similar-looking decompositions. The student correctly answered two out of four questions (Q1 and Q4), showing they can apply the concept in simpler or more abstract contexts. In Q1, they correctly identified that the decomposition $R_1 = \{A, B\},\ R_2 = \{B, C\}$ under $\Sigma = \{A \to B,\ B \to C\}$ is both lossless and dependency-preserving—this indicates a correct grasp of the intersection condition ($R_1 \cap R_2 = \{B\}$, which functionally determines R_2 via $B \to C$) and the ability to assess dependency preservation. However, in Q2, the student incorrectly chose that the decomposition does *not* have the lossless join property, when in fact $R_1 \cap R_2 = \{A\}$, and $A \to B$ and $A \to C$ imply A is a superkey, making the decomposition lossless—this reveals a gap in recognizing superkey-based losslessness. Similarly, in Q3, the student believed the decomposition was only lossless, but not dependency-preserving, while the correct answer includes both properties; this suggests confusion about whether all functional dependencies are preserved across projections, especially $B \to C$ being preserved in R_2. The one strength is in Q4, where the student correctly selected that 3NF allows for a guaranteed lossless and dependency-preserving decomposition—showing awareness of theoretical guarantees at a higher level. However, the repeated errors on structurally similar problems (Q2 and Q3) indicate unstable or incomplete internalization of the criteria for lossless joins and dependency preservation.

💭 Keep this self-awareness in mind as you work through this question.

C.2-2 Tutoring Summary

Topic: Lossless Join

=== 📚 What You Just Reviewed (Tutoring Summary)===
You recently reviewed this specific topic:
Concept Explanation:
Lossless Join Decomposition is a critical property in database normalization that ensures when a relation R is decomposed into smaller relations (e.g., R_1, R_2), joining them back using **NATURAL JOIN** will **exactly reconstruct** the original relation R without generating any **spurious (extra or incorrect) tuples**. This is essential to preserve data integrity.

A decomposition of R into R_1 and R_2 is **lossless** if and only if at least one of the following functional dependencies holds in F^+ (the closure of the functional dependencies):
- $R_1 \cap R_2 \to (R_1 - R_2)$
- $R_1 \cap R_2 \to (R_2 - R_1)$

In simpler terms: the **common attributes between** R_1 **and** R_2 must **functionally determine** all the attributes of one of the relations. Typically, this means the intersection should be a **superkey** for at least one of the decomposed relations.

Additionally, we often consider **dependency preservation**, which means that after decomposition, all functional dependencies in Σ can still be enforced on the individual relations without requiring expensive joins. While not always achievable in BCNF, it **is** guaranteed in 3NF with proper algorithms.

—

Example 1 (Question ID: 131):
$R = \{A, B, C\}, \Sigma = \{A \to B, B \to C, A \to C\}, R_1 = \{A, B\}, R_2 = \{A, C\}$

Solution:
Step 1: Check for **lossless join**.
We compute $R_1 \cap R_2 = \{A\}$
Now check whether $A \to (R_1 - R_2)$ or $A \to (R_2 - R_1)$

- $R_1 - R_2 = \{B\}$, so check: Does $A \to B$? Yes — given in Σ.
- So, $A \to B$ since A determines all of R_1's non-common part, this satisfies lossless condition.

Hence, decomposition is **lossless**.

Step 2: Check **dependency preservation**.
We need to verify if all FDs in Σ are preserved in the union of projections $\pi_{R_1}(\Sigma) \cup \pi_{R_2}(\Sigma)$

Given $\Sigma = \{A \to B, B \to C, A \to C\}$

- $A \to B$: involves A,B $\to$ both in R_1 $\to$ preserved
- $B \to C$: B,C $\to$ split across R_1 and R_2. B is in both, C only in R_2. But B is **not** a key in R_2, and more importantly, $B \to C$ **cannot be checked locally in** R_1 **or** R_2 **unless B is a superkey or determinant appears alone**.

Can $B \to C$ be inferred from the projected dependencies?

Let's project Σ onto each schema:

- $\pi_{R_1}()$: attributes {A,B}. Relevant FDs: $A \to B$. Also, since $A \to B$, and no other determinants involving just A or B, this gives us $A \to B$
- $\pi_{R_2}()$: attributes {A,C}. From $A \to C$, we get $A \to C$. Is $B \to C$ captured? No — because B is not a candidate key here, and $B \to C$ requires knowing B, but in R_2, we don't have B determining anything unless derivable.

Is $B \to C$ implied by the union of projections?

Note: We already have $A \to B$ and $A \to C$, so by transitivity $A \to C$, but $B \to C$ is an independent dependency. Can we derive $B \to C$ from others?

No — $B \to C$ is not implied by $A \to B$ and $A \to C$. For example, two rows could have same B but different C even if A values differ.

So, to enforce $B \to C$, we would need a join between R_1 and R_2, which violates dependency preservation.

Thus, **dependency** $B \to C$ **is lost**.

Therefore, the decomposition **does not preserve dependencies**.

Connection:
This example shows how a decomposition can be **lossless** because the shared attribute A functionally determines the rest (via $A \to B$), ensuring no spurious tuples upon join. However, **not all functional dependencies are preserved**, particularly $B \to C$, which cannot be enforced on either fragment alone. Hence, correct answer is **D**: fulfills lossless join but **not** dependency preserving.

—

Example 2 (Question ID: 143):
Which of the following is correct?

A. The BCNF decomposition is always unique for any R and Σ.
B. The 3NF decomposition is always unique for any R and Σ.
C. There exists an algorithm that can always generate a lossless and dependency-preserving decomposition into 3NF.
D. There exists an algorithm that can always generate a lossless and dependency-preserving decomposition into BCNF.

Solution:
Evaluate each option:

A. **False** – BCNF decomposition is **not unique**; depending on the order in which violating FDs are handled, different decompositions may result.

B. **False** – Similarly, 3NF decomposition isn't necessarily unique either; multiple valid minimal covers or choices during decomposition can lead to different results.

C. **True** – The **3NF synthesis algorithm** (also known as the **Bernstein synthesis**) guarantees:
- Lossless join
- Dependency preservation
- All relations in 3NF

This works by finding a **minimal cover** of Σ, grouping FDs by left-hand side, and creating relations accordingly.

D. **False** – It is **not always possible** to have both lossless and dependency-preserving decomposition into BCNF. Some sets of FDs **require sacrificing dependency preservation** to achieve BCNF (e.g., when there are overlapping candidate keys and non-trivial FDs).

Connection:
This question highlights a key theoretical difference between **BCNF** and **3NF**. While BCNF removes more redundancy, it **cannot guarantee** dependency preservation in all cases. In contrast, **3NF allows an algorithmic decomposition** that ensures **both lossless join and dependency preservation**, making it more practical in some scenarios where enforcing all FDs is crucial. Thus, **C is correct**.

—

Example 3 (Question ID: 130):
R = {A, B, C}, Σ = {A→B, B→C}, R_1 = {A, B}, R_2 = {B, C}

Solution:
Step 1: Check **lossless join**

- $R_1 \cap R_2 = \{B\}$
- Check if $B \to (R_1 - R_2)$ i.e., $B \to A$? Not given, and not derivable.
- Or $B \to (R_2 - R_1)$ i.e., $B \to C$? YES! Given in Σ: $B \to C$

Since $B \to C$, and $C \in R_2$, the common attribute B determines the non-common part of R_2. Therefore, the decomposition is **lossless**.

Step 2: Check **dependency preservation**

Check if all FDs in $\Sigma = \{A \to B, B \to C\}$ are preserved in the projections.

- $A \to B$: involves A,B → both in R_1. So this FD is directly enforceable in R_1
- $B \to C$: involves B,C → both in R_2. So this FD is directly enforceable in R_2

Therefore, both dependencies are preserved in their respective relations.

Also, note: No extraneous or hidden dependencies; nothing missing.

So, **both lossless join and dependency preservation hold**.

Connection:
This classic "chained" decomposition ($A \to B \to C$) splits the relation at the middle attribute B, which serves as the **link**. Since B is the key for R_2 via $B \to C$, the join is lossless. And because each FD lies entirely within one of the decomposed schemas, **all dependencies are preserved**. This exemplifies a clean, ideal decomposition satisfying **both properties**, hence answer **B**.

—

✅ Summary:
- **Lossless join** prevents data loss or corruption during decomposition.
- Always test using: intersection → difference?
- **Dependency preservation** ensures constraints can still be enforced locally.
- 3NF guarantees both; BCNF does not.
- Examples illustrate trade-offs and conditions under which these properties hold.

💡 **How to Use This Review:**

- This review is specifically about 'Lossless Join' - exactly what this question tests!
- Apply the key points and methods you just studied directly to this problem.
- Check if this question is similar to the example problems you reviewed.
- Recall the common mistakes and solution strategies you learned.

C.3 LLM RESPONSE

Task1: Yes
Task2: Lossless Join
Reasoning: This question is very similar to Example 3 (ID: 130) that I just reviewed. The relation $R = \{A, B, C\}$ is decomposed into $R1 = \{A, B\}$ and $R2 = \{B, C\}$, with $\Sigma = \{A \to B, B \to C, A \to C\}$.

First, check for **lossless join**:
- $R_1 \cap R_2 = \{B\}$
- Now check if $B \to (R_1 - R_2)$ or $B \to (R_2 - R_1)$
- $R_1 - R_2 = \{A\}$, so does $B \to A$? No, not in Σ or implied.
- $R_2 - R_1 = \{C\}$, so does $B \to C$? Yes! Given in Σ.

Since $B \to C$ holds, and C is in $R2$, the common attribute B determines the non-common part of $R2$. So, the decomposition is **lossless**.

Now check **dependency preservation**:
We need to see if all FDs in Σ can be enforced in $R1$ or $R2$ without a join.

- $A \to B$: involves A and B $\to$ both in $R1$ $\to$ preserved in $R1$
- $B \to C$: involves B and C $\to$ both in $R2$ $\to$ preserved in $R2$
- $A \to C$: this is implied by $A \to B$ and $B \to C$ (transitivity), but do we need to enforce it directly?

For dependency preservation, we don't necessarily need every given FD explicitly present, as long as the closure is preserved. But more precisely, we check if each FD in Σ is in the projection of Σ onto the decomposed schemas.

Let's project Σ:

- $_{R1}()$: attributes $\{A, B\}$. From $A \to B$ $\to$ included. $A \to C$ and $B \to C$ don't apply fully here, but $A \to B$ is captured.
- $_{R2}()$: attributes $\{B, C\}$. From $B \to C$ $\to$ included. A isn't here, so $A \to B$ and $A \to C$ not directly applicable.

But $A \to C$: since A is only in $R1$, and C only in $R2$, this FD is **split across** the two relations. Can it be enforced locally?

However, because $A \to B$ is in $R1$ and $B \to C$ is in $R2$, when we take the union of projected dependencies, we can derive $A \to C$ via transitivity. So even though $A \to C$ is not directly in either projection, it is **implied** by the combination.

But the formal definition of dependency preservation requires that the **union of the projections of Σ on $R1$ and $R2$** logically implies all FDs in Σ. That is, $(_{R1}()_{R2}())^{++}$.

In this case:
- $_{R1}()$ gives $A \to B$
- $_{R2}()$ gives $B \to C$
- Together, they imply $A \to C$

So yes, all original FDs are preserved through logical implication. Therefore, the decomposition **does preserve dependencies**.

Also, from Example 3, which had only $A \to B$ and $B \to C$, both properties held. Here we have an extra $A \to C$, but it's redundant because already implied. So adding it doesn't break anything.

Thus, both **lossless join** and **dependency preservation** are satisfied.

task3: B

PREDICTED RESULTS:
- Task1 (Self Predict): Yes
- Task2 (Know Name): Lossless Join
- Reasoning: This question is very similar to Example 3 (ID: 130) that I just reviewed. The relation $R = \{A, B, C\}$ is decomposed into $R1 = \{A, B\}$ and $R2 = \{B, C\}$, with $\Sigma = \{A \rightarrow B, B \rightarrow C, A \rightarrow C\}$.
- Task3 (Answer Choice): B

APPENDIX D: SUPPLEMENTARY VISUALISATIONS

D.1 Supplementary visualisations

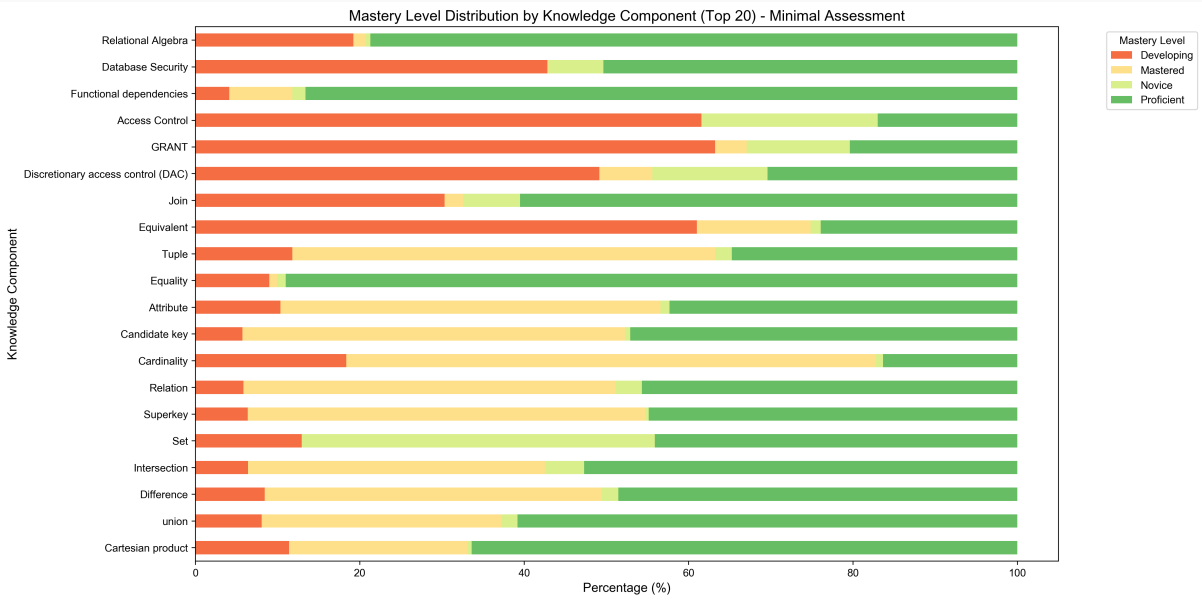

Fig. .1: Mastery level by KC minimal assessment

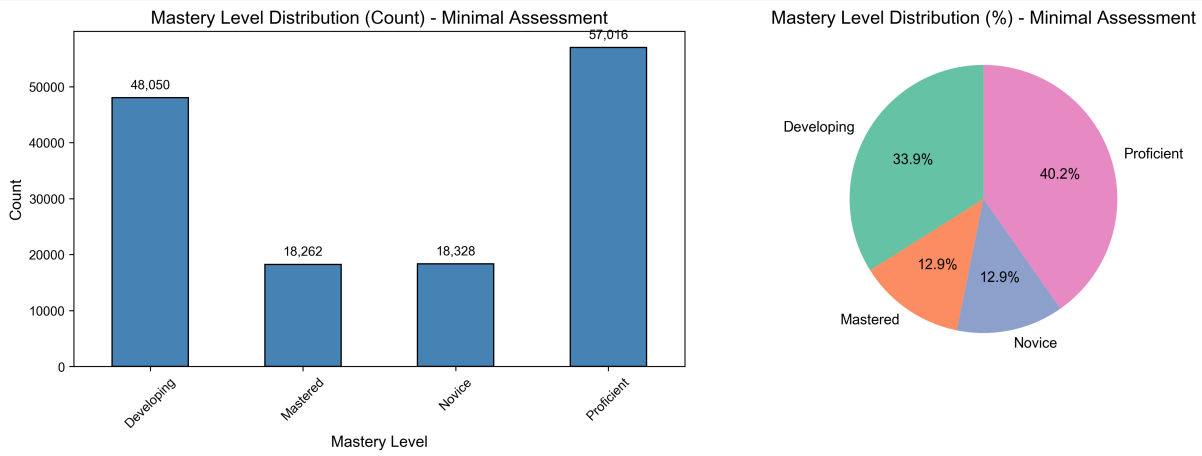

Fig. .2: Mastery level distribution (minimal assessment)